DOCUMENTATION

飞机大战程序说明文档

Python + pygame 飞机大战项目的结构剖析、实现步骤与问题分析

项目路径：D:\py\PythonProject2

运行环境：Python 3.12.10 | pygame 2.6.1（SDL 2.28.4）

文档依据：全部源码通读 + 无头（headless）实机运行验证

文档日期：2026 年 9 月 16 日

目　录

一、这个程序是做什么的 1
运行方式 1
二、文件清单与职责 1
三、程序运行的完整步骤（做什么 / 为什么 / 怎么做） 3
步骤 1：`pygame.init()` —— 初始化所有子模块 3
步骤 2：`pygame.display.set_mode((700, 700))` —— 设置游戏窗口 3
步骤 3：`pygame.time.Clock()` —— 创建游戏时钟 3
步骤 4：注册两个定时器 —— 让敌机和子弹自动出现 4
步骤 5：`create_sprite()` —— 创建精灵和精灵组 4
步骤 6：`start()` —— 游戏循环 5
四、各精灵的职责与逻辑 7
4.1 `GameSprite` —— 公共基类 7
4.2 `Background` —— 无缝滚动背景 8
4.3 `Enemy` —— 敌机 8
4.4 `Hero` —— 英雄飞机 9
4.5 `Bullet` —— 子弹 10
五、事件系统：两种输入，两种处理方式 11
六、碰撞检测 11
七、常量与素材对照表 12
事件代号 12
素材 13
关键数值 13
八、代码走查：已发现的问题 14
问题 1【严重】`Enemy.update` 缩进错误，敌机永远不销毁 14
问题 2【严重】游戏结束时 `pygame.quit()` 后循环不退出，下一帧直接崩溃 15
问题 3【中】敌机出生横向范围只有 0~500，屏幕右侧 86 像素不出敌机 15
问题 4【中】背景图被强制拉伸，画面变形 16
问题 5【中】英雄初始位置仍按 600 的基准计算 16
问题 6【低】`Hero.update` 中垂直边界钳制代码重复了一遍 16
问题 7【低】`GameSprite` 在两个文件里重复定义 16

问题 8【低】图像绘制.py 的若干遗留问题 16
问题 9【低】入口文件名带点号，无法作为模块导入 17
问题 10【提示】1.py 是纯笔记文件 17
九、学习要点回顾（对应 1.py 的笔记） 17
十、建议的修复顺序 18

说明：本目录由域代码生成。若在 Word / WPS 中编辑文档后发现页码变化，请右键单击目录并选择“更新域”（Update Field）以刷新页码。

> 项目路径：`D:\py\PythonProject2`
> 技术栈：Python 3.12.10 + pygame 2.6.1（SDL 2.28.4）
> 文档依据：全部源码通读 + 无头（headless）实机运行验证

一、这个程序是做什么的

这是一个用 pygame 写的 2D 纵向卷轴射击小游戏（俗称"飞机大战"）。玩家操纵一架英雄飞机在 700×700 的窗口里上下左右移动，游戏每秒自动生成一架敌机向下飞，英雄每 0.5 秒自动向上发射一组三连发子弹；子弹打中敌机则双方消失并计分（本版只消失不记分），敌机撞到英雄则游戏结束。背景是一张会无限向下滚动的图片，用来制造飞机在向前飞的感觉。

程序的本质是**一个由固定帧率驱动的循环**：每帧读取键盘 → 处理事件 → 做碰撞检测 → 更新所有图像位置 → 重绘屏幕。整个项目的学习重点不在于游戏玩法，而在于掌握 pygame 的四个核心概念：**游戏循环与时钟、精灵与精灵组、事件系统、碰撞检测**。

运行方式

入口文件是 `plane.start.py`，必须在项目根目录下直接运行（因为素材路径是相对路径，且文件名带点号无法被 `import`）：

```
cd D:\py\PythonProject2
.venv\Scripts\python.exe "plane.start.py"
```

在 PyCharm 里则是右键 `plane.start.py` → Run。运行后需要关闭窗口来结束程序（但见第九节问题 2，关闭窗口会报错，属于待修 bug）。

二、文件清单与职责

文件	行数	是否参与运行	作用
`plane.start.py`	133	☑ 入口	`PlaneGame` 主控类：初始化、事件监听、碰撞检测、更新绘制、游戏循环
`plane_sprite.py`	133	☑ 被入口导入	所有精灵类：`GameSprite` / `Background` / `Enemy` / `Hero` / `Bullet`，以及两个事件代号常量

文件	行数	是否参与运行	作用
enemy_sprite.py	17	❌ 未被入口使用	早期版本，只留了一个 GameSprite 基类，被 图像绘制.py 引用
图像绘制.py	63	⚠ 可独立运行	第一阶段练习脚本：只做"画背景 + 画飞机 + 背景滚动 + 两个精灵下移"，是理解原理的实验场
1.py	128	❌ 纯注释	**没有一行可执行代码**，全部是学习笔记，按主题记录了每个知识点的定义和思考顺序
image1/	—	☑ 素材目录	sz.jpg 背景、gy.png 英雄、wsc.png 敌机、ssl.png 子弹
123/	—	—	空文件夹
__pycache__/	—	—	Python 自动生成的字节码缓存，可删

依赖关系（箭头表示"导入"）：

```
plane.start.py  ──导入──▶  plane_sprite.py  ──导入──▶  pygame
                                            └──导入──▶  random

图像绘制.py      ──导入──▶  enemy_sprite.py  ──导入──▶  pygame
（独立练习，与正式游戏无耦合）

1.py             （纯注释笔记，无依赖）
```

类继承关系：

```
pygame.sprite.Sprite          （pygame 内置基类）
        ▲
        │ 继承
   GameSprite                 （本项目自建基类：加载图片、生成 rect、纵向移动）
        ▲
        ├── Background        （背景，多一个"移出屏幕后回到顶部"的重置逻辑）
        ├── Enemy             （敌机，多一个随机速度 + 随机横向位置）
        ├── Hero              （英雄飞机，重写为键盘控制 + 边界钳制 + 发射子弹）
        └── Bullet            （子弹，向上飞 + 飞出屏幕后自我销毁）
```

这种"一个基类管三件公共事（图、位置、移动），子类只写差异"的写法，是本项目最重要的设计手法。它解决的问题是：如果每个精灵都从头写加载图片、算位置、写绘制，代码会大量重复；把公共部分抽到 GameSprite 后，子类各自只剩 3~10 行真正的特殊逻辑。

三、程序运行的完整步骤（做什么 / 为什么 / 怎么做）

整个程序的生命周期就是三段：**初始化** → **建对象** → **无限循环**。下面逐步拆解。

步骤 1：`pygame.init()` —— 初始化所有子模块

```
pygame.init()                                   # plane.start.py:7
```

- **为什么**：pygame 把显示、字体、声音、计时器拆成了独立子模块，不调用 `init()` 的话后续 `set_mode()`、`Clock()`、`set_timer()` 都会报 "video system not initialized"。
- **怎么做**：一行搞定，它会把所有子模块都初始化一遍。注意 `pygame.quit()` 是它的反操作，一旦调用，后续任何显示相关操作都会立刻报错——这正是问题 2 的根源。

步骤 2：`pygame.display.set_mode((700, 700))` —— 设置游戏窗口

```
self.screen = pygame.display.set_mode((700, 700))   # plane.start.py:9
```

- **为什么**：所有绘制最终都要落在一块"画布"（surface）上，需要先确定画布大小。后续所有 `blit` 和碰撞边界判断都以这个尺寸为基准。
- **怎么做**：返回的 surface 被保存为 `self.screen`，后面每个精灵组都用 `group.draw(self.screen)` 把自己画上去。
- **注意**：窗口宽高 700 这个数字在代码里以**硬编码**形式散落在多处（背景重置的 `>= 700`、英雄边界钳制的 `700-self.rect.width`、敌机横向随机的上限 500）。这是个隐患，正确做法是定义常量 `SCREEN_W = 700`，见问题 3、5。

步骤 3：`pygame.time.Clock()` —— 创建游戏时钟

```
self.clock = pygame.time.Clock()                # plane.start.py:11
...
self.clock.tick(60)                             # plane.start.py:121（循环内）
```

- **为什么**：动画的本质是"快速连续地重画同一张图"，重画速度决定了动画是否顺滑，也决定了游戏难度（因为本项目的移动是"每帧移动 N 像素"，帧率越高飞机跑得越快）。`Clock` 就是用来把重画速度**锁定**在固定频率上的。
- **怎么做**：在循环开头调用 `tick(60)`，它做了两件事：① 让本帧至少耗时 1/60 秒（不足则自动休眠等待），使游戏在任何性能的电脑上都以相同速度运行；② 返回距上一帧的毫秒数（本项目没用到返回值）。

- **为什么要写在循环开头**：因为它是"控制整圈循环节奏"的，写在哪都会生效，但放开头最符合"先定节奏、再干活"的语义。

步骤 4：注册两个定时器 —— 让敌机和子弹自动出现

```
pygame.time.set_timer(plane_sprite.ENEMY_EVENT, 1000)   # plane.start.py:14
pygame.time.set_timer(plane_sprite.FIRE, 500)           # plane.start.py:17
```

对应 `plane_sprite.py` 中定义的事件代号：

```
ENEMY_EVENT = pygame.USEREVENT        # 实测值 32866
FIRE = pygame.USEREVENT + 1           # 实测值 32867
```

- **为什么不用"数帧"的方式**（比如"每 60 帧生成一架敌机"）？因为数帧写法和帧率强耦合，一旦 `tick(60)` 改成 `tick(30)`，敌机出场的实际时间间隔就会翻倍，游戏难度全乱。`set_timer` 用的是**真实毫秒**，与帧率解耦，改帧率不会影响出怪节奏。
- **为什么要自己定义事件代号**？pygame 的事件类型是一个整数枚举，内置事件（如 `QUIT`）已经占用了小数字。`pygame.USEREVENT` 是官方留出的"自定义事件起点"，自己定义的事件必须基于它往后加，避免和内置事件撞号。
- **怎么做**：`set_timer(代号, 间隔毫秒)` 只是向 pygame 的计时器**登记**，真正的效果是"每隔 1000 毫秒，pygame 会往事件队列里塞一个类型为 `ENEMY_EVENT` 的事件对象"。它本身不执行任何游戏逻辑——逻辑在步骤 6 的事件监听里。
- **为什么"发射子弹"也做成定时器**？因为需求是"英雄自动开火，玩家只控制移动"。自动行为用定时器最自然。

步骤 5：`create_sprite()` —— 创建精灵和精灵组

```
self.create_sprite()                                    # plane.start.py:21
```

在 `__init__` 里单独抽成一个方法，是为了让初始化代码保持"一句话一步"的可读性。这一步做的事：

① 创建两组背景精灵，实现无缝滚动

```
bg1 = plane_sprite.Background("image1/sz.jpg", size=(700, 700))
bg2 = plane_sprite.Background("image1/sz.jpg", size=(700, 700))
bg2.rect.y = -bg2.rect.height        # 第二张图挪到屏幕上方外（y = -700）
```

- **为什么是两张**：一张图往下滚，滚到底部后会露出空白。用两张同样的图上下首尾相接，当上面那张完全移出屏幕底部时，把它瞬间挪回屏幕顶部（`y = -height`），就能无缝循环。原理类似扶梯梯级：梯级本身是循环的，乘客却感觉一直在前进。

- **为什么要 `size=(700,700)`**：原图 `sz.jpg` 是 1263×977，比窗口大，必须缩放，否则只能看到左上角一块。

② 创建三个精灵组

```
self.bg_group = pygame.sprite.Group()
self.bg_group.add(bg1, bg2)
self.enemy_group = pygame.sprite.Group()            # 敌机组，创建时为空
self.hero = plane_sprite.Hero()
self.hero_group = pygame.sprite.Group()
self.hero_group.add(self.hero)
```

- **为什么用精灵组**：Group 提供了三个批量操作，让主循环不必知道细节：
- `group.update()` —— 遍历调用组内每个精灵的 `update()`（各自按自己的规则移动）；
- `group.draw(screen)` —— 遍历把每个精灵的 `image` 画到 `rect` 指定的位置；
- `groupcollide()` —— 成批做两两碰撞检测。

如果没有组，主循环里就得写 `for` 循环手动遍历、手动判断哪些精灵已经飞出屏幕要跳过。用组之后，"敌机从 0 架变成 20 架"这件事对主循环**完全透明**。

- **为什么英雄也要单独建组**：英雄只有一架，看似不需要组，但精灵组是 `groupcollide` 的入参类型，且碰撞后英雄可以被 `kill()` 移出组；同时"英雄组"和"敌机组"在语义上对称，便于统一处理。
- **`add()` 只是把精灵挂进组，并不会把它画出来**；真正的绘制发生在主循环（步骤 6 第 5 小步）。

步骤 6：`start()` —— 游戏循环

这是整个程序的心脏，一个 `while True` 永不退出的循环。**所有游戏都是这个结构**："每一帧都重复做同样几件事"。

```
while True:
    self.clock.tick(60)        # ① 定节奏：锁定 60 帧/秒
    self.event_handler()       # ② 听输入：键盘 + 定时器事件
    self.collide()             # ③ 判碰撞：子弹打敌机 / 敌机撞英雄
    self.update_sprite()       # ④ 更新 + 绘制所有精灵
    pygame.display.update()    # ⑤ 把这一帧的结果显示到屏幕上
```

① `tick(60)`：为什么必须放在循环里——它管的是"每帧之间的间隔"，只有循环往复才需要控制节奏。

② `event_handler()`：监听用户和系统的一切动静。本方法做了两件性质完全不同的事：

```
# A. 轮询按键状态 —— 处理"持续按住"
```

```
keys = pygame.key.get_pressed()
if keys[pygame.K_RIGHT]:
    self.hero.speed = 3
elif keys[pygame.K_LEFT]:
    self.hero.speed = -3
else:
    self.hero.speed = 0
if keys[pygame.K_UP]:
    self.hero.upanddown = -2
elif keys[pygame.K_DOWN]:
    self.hero.upanddown = 2
else:
    self.hero.upanddown = 0
```

- **为什么用 `get_pressed()` 而不是事件**：玩家按住方向键时，操作系统只在**按下瞬间**发一次按键事件，之后不会持续发。而移动需要"按住期间一直移动"。所以这里用 `get_pressed()` 每帧去**询问当前哪些键是按下状态**（返回一个元组，被按下的键对应位置为 1，用 `pygame.K_RIGHT` 这样的常量做下标去查）。
- **为什么用"设置速度"而不是"直接改坐标"**：这里只把速度赋给 `hero.speed` / `hero.upanddown`，**不移动位置**。位置的更新留给 `Hero.update()`。这是 MVC 式的职责分离——事件层只表达"意图"，精灵层才负责"执行"。这样做的好处是：松开键时速度归 0，飞机立刻停下，不会因为事件漏收而"卡住一直飞"。
- **注意细节**：左右和上下是**两个独立的 if 块**（不是 elif 串联），所以斜向移动可以同时生效——按"右上"时水平速度 3、垂直速度 -2，飞机沿对角线飞。
- **`else: speed = 0`** 保证了没人按键时飞机不漂移（默认速度实测为 0，来自 `Hero` 构造时的 `super().__init__('image1/gy.png', 0)`）。

```
# B. 遍历事件队列 —— 处理"一次性动作"
for event in pygame.event.get():
    if event.type == plane_sprite.ENEMY_EVENT:
        enemy = plane_sprite.Enemy()
        self.enemy_group.add(enemy)                # 造一架敌机丢进敌机组
    elif event.type == pygame.QUIT:
        pygame.quit()                              # 关闭窗口
    elif event.type == plane_sprite.FIRE:
        self.hero.fire()                           # 发射三连发
```

- **为什么这次要用事件**：定时器到点、用户点关闭按钮，这些都是**瞬时的、一次性的**动作，不是"持续状态"。`pygame.event.get()` 返回自上次调用以来累积的所有事件对象列表，遍历它就能捕捉到"刚才发生了什么"。
- **`event_handler` 为什么能"监听定时器"**：因为 `set_timer` 登记的定时器到点后会**往这个队列里塞事件**，所以第 4 步登记的定时器，在这里才真正变成游戏行为。这就是"定时器负责报时、事件循环负责干活"的分工。
- **注意**：`pygame.event.get()` 会**清空**队列，所以每帧调用一次刚好，调用两次会导致第二次拿到空列表。

③ **collide()：碰撞检测**（第 7 节详解）

④ **update_sprite()：先更新位置，再绘制**

```
self.bg_group.update();     self.bg_group.draw(self.screen)
self.enemy_group.update();  self.enemy_group.draw(self.screen)
self.hero_group.update();   self.hero_group.draw(self.screen)
self.hero.bullet_group.update(); self.hero.bullet_group.draw(self.screen)
```

- **为什么四个组要分组处理、且顺序固定**：update 和 draw 必须成对。顺序即**图层顺序**——后 draw 的会盖住先 draw 的。这里背景在最底，然后是敌机，最后英雄和子弹在最上层，遮挡关系符合视觉预期。
- **为什么子弹组挂在英雄身上**（self.hero.bullet_group）而不是放在 PlaneGame 里：子弹是由英雄发射的，把子弹组作为英雄的一个属性（组合关系）在语义上更贴切，也让"英雄死则子弹无处可发"这种关联更自然。缺点是主循环要写成 self.hero.bullet_group.xxx，稍长。

⑤ **pygame.display.update()：把这一帧真正显示出来**

- **为什么必须最后调用、且整圈只调用一次**：前面所有的 blit（把图片画到画布）都只发生在**内存里的画布**上，屏幕还是上一帧的样子。display.update() 才是"提交"动作，把内存画布一次性推到屏幕上。它有"多张图互相覆盖"的效果——所有绘制都完成后统一提交一次，屏幕上看到的就是最终结果。如果每画一张图就 update 一次，会因为频繁提交而闪烁、性能差。

四、各精灵的职责与逻辑

4.1 GameSprite —— 公共基类

```
class GameSprite(pygame.sprite.Sprite):
    def __init__(self, image_name, speed=1, size=None):
        super().__init__()                          # Sprite 基类构造无参数
        self.image = pygame.image.load(image_name)  # 图像数据
        if size is not None:
            self.image = pygame.transform.scale(self.image, size)
        self.rect = self.image.get_rect()           # 位置+尺寸对象，默认 (0,0)
        self.speed = speed

    def update(self, *args):
        self.rect.y += self.speed                   # 垂直向下运动
```

- **为什么必须有 image 和 rect 这两个固定名字的属性**：pygame.sprite.Group.draw() 内部就是读 sprite.image 和 sprite.rect 来

绘制的，groupcollide() 也是读 sprite.rect 来做矩形相交判断。名字写错任何一个，精灵组就"看不见"这个精灵。

- **super().__init__() 为什么不能省**：pygame.sprite.Sprite 的构造函数会给精灵挂上 _Sprite__g（所属组）等内部属性，不调用它，后续 group.add() 可能不生效。
- **get_rect() 的作用**：把图片的宽高转成一个 Rect 对象（带 x, y, width, height, left, right, center, centerx, centery 等几何属性），这样才能方便地做移动和碰撞。默认 x、y 都是 0。
- **update(self, *args) 里的 *args 是干什么的**：Group.update() 和 Group.update(*args) 都会把调用时带的参数**原样转发**给每个精灵的 update()。基类写成 *args 是为了"不关心、也不会被子类传参时报错"。目前项目里没用到传参，但这是 pygame 精灵的标准写法。
- **注意 speed 的正负号约定**：rect.y += speed，所以**正数向下**（背景、敌机用正数），**负数向上**（子弹用 -2）。而英雄的 speed 被复用为**水平**速度（在 Hero.update 里是 rect.x += speed），这是同一个属性名在不同子类里含义不同，属于可读性上的小瑕疵（更清晰的做法是拆成 speed_x / speed_y）。

4.2 Background —— 无缝滚动背景

```
class Background(GameSprite):
    def update(self):
        super().update()                            # 先按父类向下移动 speed=1 像素
        if self.rect.y >= 700:                      # 移出屏幕底部
            self.rect.y = -self.rect.height         # 挪到屏幕顶部外侧，形成循环
```

- **为什么 update() 不带 *args 也不会报错**：因为主循环调用的是 self.bg_group.update()，不传参数；pygame 转发时也是零参数。
- **为什么阈值是 700**：屏幕高度是 700，rect.y >= 700 意味着这张图的**顶边**已经滑到屏幕下方了，此时它完全不可见，正是搬家到顶部的最佳时机。
- **实现效果**：bg1 从 y=0 开始、bg2 从 y=-700 开始，两者都以每帧 1 像素向下移动；当 bg1 到 700 时跳回 -700，此时 bg2 刚好在 0 的位置接上。如此交替，形成无限滚动的视觉（已实测验证重置正确）。

4.3 Enemy —— 敌机

```
class Enemy(GameSprite):
    def __init__(self):
        super().__init__('image1/wsc.png')
        self.speed = random.randint(2, 4)     # 每架敌机的下落速度都不同
```

```
        self.rect.x = random.randint(0, 500)  # 横向随机出生点
```

- **为什么速度随机**：如果所有敌机同速下落，几十架敌机在视觉上会排成一条平直的横线，很假。2~4 的随机速度让它们互相穿插，观感自然得多。
- **为什么横向位置随机**：让玩家不能只守一个位置。
- **注意 `rect.y` 没有随机**：敌机一律从 y=0（屏幕顶部）出生，这是"敌机从上方飞入"的必要条件。
- **注意 `random.randint(0, 500)` 的上限 500 是个 bug**：敌机宽 114 像素，出生点最多到 500 说明右边缘 614，屏幕最右侧 86 像素永远不会有敌机出现。见问题 3。

4.4 Hero —— 英雄飞机

```
class Hero(GameSprite):
    def __init__(self):
        super().__init__('image1/gy.png', 0)   # speed 初值 0，由键盘改写
        self.rect.x = 600 - self.rect.width    # 初始 x = 505
        self.rect.y = 600 - self.rect.height   # 初始 y = 475
        self.upanddown = 0                     # 垂直速度，由键盘改写
        self.bullet_group = pygame.sprite.Group()   # 自己的子弹组

    def update(self):
        self.rect.x += self.speed
        self.rect.y += self.upanddown
        if self.rect.x >= 700 - self.rect.width:
            self.rect.x = 700 - self.rect.width
        elif self.rect.x <= 0:
            self.rect.x = 0
        if self.rect.y >= 700 - self.rect.height:
            self.rect.y = 700 - self.rect.height
        elif self.rect.y <= 0:
            self.rect.y = 0
```

- **为什么英雄不用 `GameSprite.update()`**：父类的行为是"按 speed 向下移动"，而英雄要"水平和垂直分别按不同速度移动"，属于完全不同的运动模型，所以整个 `update` 重写（注意 `super().update()` 一次都没调用）。
- **为什么"先移动、再钳制"**：如果只判断不钳制，飞机飞出屏幕后就再也回不来了（因为 `rect.x` 会一直累加，而画面外无法操作）。所以每次移动后立刻检查四条边，越界就把坐标"压"回边界上，形成"撞到屏幕边缘停住"的手感。
- **为什么边界是 `700 - self.rect.width` 而不是 700**：`rect.x` 是**左上角**坐标。要让**右边缘**贴上屏幕右边界，左上角最多只能到 `700 - 宽度`（实测英雄宽 95，所以最大 x = 605）。写 `700` 会让飞机半个身子跑到屏幕外。
- **为什么要引入 `upanddown` 这个新属性**：因为父类的 `speed` 已经被用来表示水平速度了，垂直速度必须另找一个属性名。这样做到了"左右移动和上下移动互不干扰"，可以斜飞、也可以只按一个方向。

- **初始位置 `600 - width/height` 的问题**：窗口是 700×700，但初始位置却按 600 的基准算，导致英雄出现在 (505, 475)——偏右下而不是底部居中。见问题 5。

发射子弹：

```
def fire(self):
    for i in range(3):                          # 一次连发三颗
        bullet = Bullet()
        bullet.rect.centerx = self.rect.centerx   # 水平方向与飞机中心对齐
        bullet.rect.y = self.rect.y - (i + 1) * 20   # 依次向上错开 20 像素
        self.bullet_group.add(bullet)
```

- **为什么用 `centerx` 对齐而不是 `x`**：`rect.x` 是左边缘，子弹宽度（12）和飞机宽度（95）差很多，用左边缘对齐会让子弹明显偏左。用 `centerx`（中心线）对齐，子弹才会从机头正中间射出。
- **为什么 `y` 要减 `(i+1)*20`**：三颗子弹如果都在同一个 y，会完全重叠、看上去只有一颗。让第 1 颗在飞机上方 20 像素、第 2 颗 40、第 3 颗 60，就形成了"一串三发"的视觉效果（实测三颗的 y 为 455 / 435 / 415，飞机 y = 475）。
- **为什么 `Bullet()` 创建后立刻 `add()` 到组里**：只有进了组，主循环的 `bullet_group.update()` 和 `draw()` 才会管它。不进组的精灵相当于"创建了但没人管"，位置永远不动、也永远不显示。
- **每次 `fire()` 都会新建 3 个对象**——这正是为什么子弹的"飞出屏幕就销毁"如此重要，否则对象会无限累积。

4.5 `Bullet` —— 子弹

```
class Bullet(GameSprite):
    def __init__(self):
        super().__init__('image1/ssl.png', -2)   # 速度为负 → 向上飞

    def update(self):
        super().update()                          # 复用父类的 rect.y += speed
        if self.rect.y <= 0:                      # 飞出屏幕顶部
            self.kill()                           # 从所属的精灵组里删除自己
```

- **为什么可以复用父类的 `update`**：子弹的运动是"垂直直线匀速"，和父类 `rect.y += speed` 完全一致，只是方向相反（速度为 -2）。这体现了基类设计的价值——子类只需改**参数**，不用改**算法**。
- **为什么 `kill()` 是必要的**：如果子弹飞出屏幕后不删，主循环每帧仍然会去更新它的位置、绘制它（画在屏幕外的无效区域），对象和计算量会随时间无限增长，游戏越玩越卡。`kill()` 是 `Sprite` 提供的自我移除方法，它会把精灵从**所有**所属组里摘掉，之后 `update/draw` 就不再遍历它。这就是"垃圾回收"。

- **同样的问题在 `Enemy` 里也存在**，而且更为严重——见问题 1。

五、事件系统：两种输入，两种处理方式

这是本项目最容易被初学者混淆、但设计上最讲究的一点。程序同时处理**两类完全不同的输入**，用了**两套完全不同的 API**：

	键盘移动	敌机出场 / 自动开火
输入性质	**持续状态**："现在是否被按住"	**离散动作**："刚刚发生了一次"
用的 API	`pygame.key.get_pressed()`（**轮询 / 主动询问**）	`pygame.event.get()`（**事件队列/被动接收**）
触发频率	每帧查询一次，按住期间持续为真	只在事件发生时产生一次事件对象
存放位置	不存队列，每次返回当前**快照**	存在队列里，`get()` 后**清空**
代码里改什么	改 `hero.speed`（意图）	直接调用 `hero.fire()` / `enemy_group.add()`（执行）

判断依据很简单：**问自己"这个操作需要按住时持续生效吗？"** 需要 → 轮询；不需要、只是一次性的 → 事件。方向键移动属于前者，定时器到点属于后者，退出按钮也属于后者。

六、碰撞检测

```
def collide(self):
    # 子弹撞毁敌方飞机：子弹和敌机都消失
    pygame.sprite.groupcollide(self.hero.bullet_group, self.enemy_group, True, True)

    # 敌方飞机撞英雄飞机：游戏结束
    ret = pygame.sprite.groupcollide(self.hero_group, self.enemy_group, True, True)
    if ret:
        pygame.quit()
```

`groupcollide(组 1, 组 2, dokill1, dokill2)` 的参数含义：

参数	含义
组 1、组 2	要两两比对的两个精灵组
dokill1	组 1 中**发生碰撞**的精灵是否自动 `kill()` 删除
dokill2	组 2 中**发生碰撞**的精灵是否自动 `kill()` 删除

- **返回值**：发生了碰撞时返回一个字典，**键是组 1 的精灵、值是组 2 中被它撞到的精灵列表**（如 `{敌机: [英雄]}`）；没有任何碰撞则返回空字典 `{}`——正好可以当布尔值用于 `if ret:` 判断"游戏是否结束"。
- **为什么子弹和敌机的碰撞没有用返回值**：因为规则是"双方都消失"，`True, True` 已经自动完成了，返回值用不上。
- **为什么英雄的碰撞要用返回值**：因为规则不是"两个都消失"，而是要触发**游戏结束**这个额外动作，必须拿到"有没有撞上"这个信息。实测：1 颗子弹 + 5 架敌机 → 0 颗子弹 + 4 架敌机（子弹与敌机正确配对销毁）；敌机压到英雄身上 → 英雄组和敌机组都清空。
- **注意**：英雄碰撞这里用了 `True, True`，意味着**英雄自己也会被删掉**（退出前英雄组就空了）。如果以后要做"英雄爆炸动画"或"扣血重生"，这里应该改成 `False, False` 并自行处理销毁时机。
- **注意**：`pygame.quit()` 之后循环并没有停，这是问题 2。

七、常量与素材对照表

事件代号

常量	定义	实测值	用途
`ENEMY_EVENT`	`pygame.USEREVENT`	32866	敌机出场定时器代号，间隔 1000 ms
`FIRE`	`pygame.USEREVENT + 1`	32867	英雄开火定时器代号，间隔 500 ms

素材

文件	原始尺寸	代码中的缩放	用途	被谁加载
image1/sz.jpg	1263 × 977	→ 700 × 700	滚动背景	Background
image1/gy.png	95 × 125	不缩放	英雄飞机	Hero
image1/wsc.png	114 × 128	不缩放	敌机	Enemy
image1/ssl.png	12 × 25	不缩放	子弹	Bullet
image1/back1.jpg、plane1.webp	—	—	备用素材，当前未使用	—

关键数值

项目	数值	位置
窗口尺寸	700 × 700	plane.start.py:9
帧率	60 fps	plane.start.py:121
敌机出场间隔	1000 ms	plane.start.py:14
开火间隔	500 ms	plane.start.py:17
英雄水平速度	±3 像素/帧	plane.start.py:48,50
英雄垂直速度	±2 像素/帧	plane.start.py:55,57
敌机下落速度	2~4 像素/帧（随机）	plane_sprite.py:36
敌机出生横向范围	x ∈ [0, 500]	plane_sprite.py:38

项目	数值	位置
子弹速度	−2 像素/帧（向上）	`plane_sprite.py:97`
单次发射子弹数	3 颗，间距 20 像素	`plane_sprite.py:79-83`
背景滚动速度	1 像素/帧	`GameSprite` 默认 `speed=1`

八、代码走查：已发现的问题

以下问题全部经过**实机运行验证**（无头模式 SDL 驱动），不是静态猜测。

问题 1【严重】`Enemy.update` 缩进错误，敌机永远不销毁

位置：`plane_sprite.py:41-45`

```
class Enemy(GameSprite):
    def __init__(self):
        ...
        def update(self):        # ← 缩进多了 4 格，被定义在 __init__ 内部!
            if self.rect.y >= 700:
                self.kill()
```

现象：敌机飞出屏幕后不会消失，对象和绘制开销无限累积，游戏运行越久越卡。 **原因**：`def update` 缩进在 `__init__` 内部，它只是**每次创建敌机时定义了一个内部的局部函数，然后从来没有调用过它就丢掉了**。因此 `Enemy` 实际继承的是 `GameSprite.update()`（只做 `rect.y += speed`）。 **验证**：`'update' in Enemy.__dict__` 返回 `False`，`Enemy.update is GameSprite.update` 返回 `True`；生成 5 架敌机并跑 400 帧后，`enemy_group` 里仍是 5 架，y 坐标分别为 800 / 1200 / 1600 / 800 / 1200（屏幕高仅 700，它们早就在屏幕外了）。 **修复**：把 `def update` 的缩进减 4 格，与 `def __init__` 对齐即可：

```
class Enemy(GameSprite):
    def __init__(self):
        super().__init__('image1/wsc.png')
        self.speed = random.randint(2, 4)
        self.rect.x = random.randint(0, 500)

    def update(self):                       # 与 __init__ 同级
        super().update()                    # 先做父类的向下移动
        if self.rect.y >= 700:
            self.kill()
```

> 顺带说明：原代码里 `Enemy.update` 也**没有调用 `super().update()`**，如果只是修正缩进而不补上这一行，敌机会当场卡住原地不动——所以这两处必须一起改。

问题 2【严重】游戏结束时 pygame.quit() 后循环不退出，下一帧直接崩溃

位置：`plane.start.py:94-96`（`collide`）+ `plane.start.py:118`（`while True`）

现象：敌机撞到英雄时，程序不是干净退出，而是抛异常崩溃，控制台打印 `pygame.error: video system not initialized`。 **原因**：`collide()` 里调用了 `pygame.quit()`，它关闭了所有 pygame 子模块；但 `while True` 没有任何退出条件，循环随即进入下一帧并调用 `self.clock.tick(60)`、`self.event_handler()`，而这些操作都依赖已经关闭的显示系统。 **验证**：把敌机放到英雄位置后调用 `collide()`，`pygame.get_init()` 变为 `False`，紧接着执行一帧游戏循环，立刻抛出 `error - video system not initialized`。 **修复**：用一个标志位让循环体自己退出，而不是在深处直接 `quit()`：

```
class PlaneGame:
    def __init__(self):
        ...
        self.is_running = True                    # 新增

    def event_handler(self):
        ...
        elif event.type == pygame.QUIT:
            self.is_running = False               # 改：只置标志

    def collide(self):
        pygame.sprite.groupcollide(self.hero.bullet_group, self.enemy_group, True, True)
        ret = pygame.sprite.groupcollide(self.hero_group, self.enemy_group, True, True)
        if ret:
            self.is_running = False               # 改：只置标志

    def start(self):
        while self.is_running:                    # 改：由标志控制
            self.clock.tick(60)
            ...
        pygame.quit()                             # 循环真正结束后再统一退出
```

问题 3【中】敌机出生横向范围只有 0~500，屏幕右侧 86 像素不出敌机

位置：`plane_sprite.py:38` **原因**：敌机宽 114 像素，上限 500 使敌机右边缘最多到 614，屏幕宽 700。硬编码的 500 应是早期版本的遗留值。 **修复**：`self.rect.x = random.randint(0, 700 - self.rect.width)`（或改用常量 `SCREEN_W`）。

问题 4【中】背景图被强制拉伸，画面变形

位置：`plane.start.py:27,29` → `size=(700, 700)` **原因**：原图 1263 × 977（宽高比约 1.29）被压成 700 × 700（宽高比 1.0），也就是说画面被**纵向压扁**了。**修复**：要么按原比例缩放后居中（用 `pygame.transform.smoothscale` 配合裁剪），要么把窗口尺寸改成与原图比例一致。若只是学习框架，可暂不处理，但要意识到这不是"图片不好看"而是代码把比例改掉了。

问题 5【中】英雄初始位置仍按 600 的基准计算

位置：`plane_sprite.py:52-53` **原因**：`self.rect.x = 600 - self.rect.width`、`self.rect.y = 600 - self.rect.height`，但窗口已经是 700 × 700。实测英雄出现在 (505, 475)，位于屏幕偏右中部，而不是"底部居中"这一射击游戏的惯常出场地。 **修复**：改为按窗口尺寸计算，例如水平居中、垂直靠底：

```
self.rect.centerx = 700 // 2
self.rect.bottom = 700 - 20
```

问题 6【低】Hero.update 中垂直边界钳制代码重复了一遍

位置：`plane_sprite.py:68-75` **原因**：第 68-71 行和第 72-75 行是完全相同的四个判断，复制粘贴时留下的冗余。功能上无害（第二次判断时值已经合法），但会让读者误以为两段有区别。 **修复**：删掉第 72-75 行。

问题 7【低】GameSprite 在两个文件里重复定义

位置：`enemy_sprite.py:4-15` 与 `plane_sprite.py:9-20` **原因**：`图像绘制.py` 写在前，从 `enemy_sprite.py` 导入基类；后来做正式游戏时又在新文件里重写了一份，旧文件就成了孤儿。两份实现目前完全一致，一旦以后只改了其中一份，就会出现"同一类名、两种行为"的隐蔽 bug。 **修复**：删除 `enemy_sprite.py`，让 `图像绘制.py` 也从 `plane_sprite.py` 导入 `GameSprite`（它已经在 `plane_sprite.py` 里了）。

问题 8【低】图像绘制.py 的若干遗留问题

位置：`图像绘制.py:7,11,20,22` 等

- **素材扩展名大小写混用**：文件实际叫 `sz.jpg` / `gy.png` / `ssl.png` / `wsc.png`，代码里写的是 `.JPG`。Windows 文件系统不区分大小写所以能跑，**一旦把项目拷到 Linux/macOS 就会全部 "No such file or directory" 报错**。

- **没有处理 `QUIT` 事件**：关闭窗口不会退出进程，窗口变黑屏卡住，只能用任务管理器结束。循环条件是 `while True`，应加退出条件。
- **`plane_rect.y = 600` 超出窗口**：窗口只有 700 × 400，飞机"从下方重新进入"时被设在 y=600，会有一段完全看不见的空档。
- 该文件同时保留了"用 rect 直接画图"和"用精灵组管理"两种写法，是刻意的教学对照，阅读时不要把两者当成同一套机制。

问题 9【低】入口文件名带点号，无法作为模块导入

位置：文件名 `plane.start.py` **现象**：`import plane.start` 抛出 `ModuleNotFoundError: No module named 'plane'`。**原因**：Python 模块名不允许含点号（点表示包层级），`plane.start` 会被理解成"`plane` 包里的 `start` 子模块"。目前只能**直接运行**该文件（此时 `__name__ == "__main__"`，`import plane_sprite` 正常工作），PyCharm 的 Run 按钮也是这个路径，所以问题没有暴露。**修复**：重命名为 `plane_start.py` 或 `plane_main.py`（PyCharm 会同步提示修改引用）。

问题 10【提示】`1.py` 是纯笔记文件

`1.py` 全部 128 行都是 `#` 注释，**没有任何可执行语句**。它不是程序的一部分，而是学习过程中按知识点顺序记的笔记（初始化 → 游戏循环 → 精灵 → 定时器 → 事件 → 碰撞）。保留它完全没问题，但需要知道它不参与运行，运行时不会有任何输出。

九、学习要点回顾（对应 `1.py` 的笔记）

`1.py` 中的笔记主题	在代码中的落地位置
`pygame.init()` / `Rect` / `set_mode` / `blit` / `display.update()` 五步曲	`plane.start.py:7-11`、`图像绘制.py:3-17`
"所有 blit 完成后统一调用一次 update"	`plane.start.py:130`（循环末尾唯一一次）
游戏循环 + 游戏时钟 + 帧的概念（每秒 60 帧）	`plane.start.py:117-130`
"飞出屏幕顶部后从下方重新进入"	`Background.update`（反向：从顶部重新进入）

1.py 中的笔记主题	在代码中的落地位置
事件 / 监听 / pygame.event.get() 返回事件列表	plane.start.py:62
精灵与精灵组（image / rect / update、add / update / draw）	plane_sprite.py:9-20 、plane.start.py:98-111
敌机出场：set_timer(事件代号, 毫秒) + USEREVENT	plane_sprite.py:5,7 、plane.start.py:14,64
英雄飞机唯一、不能自己移动、定时发射三连发	plane_sprite.py:48-86 、plane.start.py:17,75-77
键盘控制：pygame.key.get_pressed() 返回元组、按键常量作下标	plane.start.py:45-59
Rect 的 x/y/width/height/left/right/center	全项目广泛使用（如 bullet.rect.centerx）
子弹飞出屏幕后删除	Bullet.update + self.kill()
碰撞检测 groupcollide（dokill1/dokill2、返回字典 {组 1: 组 2}）	plane.start.py:87-96

十、建议的修复顺序

按"影响正确性 → 影响体验 → 影响整洁"排列，前三项建议优先处理：

1. **问题 1**（敌机不销毁）—— 改缩进 + 补 super().update()，两行改动，直接解决游戏越玩越卡。
2. **问题 2**（退出即崩溃）—— 用 is_running 标志替换直接 pygame.quit()，让程序能正常关闭窗口退出。
3. **问题 3**（右侧不出敌机）—— 一行改动，让游戏难度分布合理。
4. **问题 5、6**（英雄位置、重复代码）—— 影响观感和可读性。
5. **问题 4、8、9**（图片比例、旧脚本、文件重命名）—— 属于整洁性与跨平台兼容性，可在学习下一阶段时顺手处理。

修复 1、2 之后，这个游戏就能完整地"开局 → 打敌机 → 撞机 → 干净退出"跑通一整个生命周期了。